BrandCraft : Gen AI–Powered Branding Automation System

PROJECT DESCRIPTION:

BrandCraft is an AI-powered branding assistant designed to help users create and develop a complete brand identity. Traditionally, creating a brand involves multiple stages such as selecting a brand name, designing a logo, writing marketing content, analyzing customer feedback, and seeking branding consultation. These tasks usually require different tools, professional designers, marketers, and significant financial investment.

To address these challenges, BrandCraft integrates Generative AI technologies into a single platform capable of performing all these activities automatically.

The system allows users to:

- Generate creative brand names based on industry and business requirements.
- Create professional taglines and brand descriptions.
- Design AI-generated logos using text prompts.
- Generate marketing content such as advertisements, social media captions, product descriptions, and email campaigns.
- Analyze customer reviews and sentiments.
- Interact with an AI-powered branding assistant for strategic branding guidance.
- Store and manage all generated branding assets within the platform.

The application is developed using FastAPI for backend services, HTML, CSS, and JavaScript for the frontend interface, and SQLite as the database. The platform communicates with external AI services such as Google Gemini API and Hugging Face Stable Diffusion XL models to generate intelligent outputs.

The project demonstrates the practical implementation of Generative AI in branding and digital marketing by transforming complex branding workflows into an automated, user-friendly solution.

SCENARIOS:

Scenario 1: Startup Founder Building a New Brand

A startup founder wants to launch a new technology company but has not finalized a brand identity. Creating a brand manually would require marketing consultants, designers, and content writers.

BrandCraft generates:

- Multiple AI-generated brand name suggestions
- Unique taglines and slogans
- Brand meaning and positioning statement
- Professional AI-generated logo designs
- Complete brand story and brand description

Scenario 2: Entrepreneur Creating Marketing Content

A business owner launching a new product needs promotional content for social media, advertisements, and product pages but lacks content-writing expertise.

BrandCraft generates:

- Product descriptions
- Social media captions
- Advertisement copy
- Email marketing content
- Promotional slogans and campaign ideas

Scenario 3: Business Owner Designing a Professional Logo

A small business owner needs a visually appealing logo for their brand without hiring a graphic designer.

BrandCraft generates:

- AI-powered logo concepts
- Multiple logo variations
- Customized styles and color themes
- High-quality branding visuals
- Downloadable logo assets

Scenario 4: Marketing Team Analyzing Customer Feedback

A marketing team receives hundreds of customer reviews and wants to understand customer opinions regarding their products and services.

BrandCraft performs:

- Sentiment analysis on customer reviews
- Positive, negative, and neutral classification
- Emotion detection
- Keyword extraction
- Customer perception summary report

Scenario 5: Startup Seeking Branding Consultation

A newly established startup is unsure about branding strategies, color selection, audience targeting, and brand positioning.

BrandCraft's AI Assistant provides:

- Branding recommendations
- Color palette suggestions
- Brand personality guidance
- Target audience insights
- Marketing and growth strategies

ARCHITECTURE OVERVIEW:

FastAPI Backend
Handles API routing, business logic processing, user authentication, database operations, and communication with external AI services.

Frontend (HTML + CSS + JavaScript)
Provides a responsive and interactive user interface with fetch-based API communication for seamless user interaction.

AI Models Integrated

Google Gemini API → Brand name generation, tagline creation, marketing content generation, sentiment analysis, and AI branding assistance.

Stable Diffusion XL (SDXL) → AI-powered logo generation and brand visual creation.

SQLite Database with SQLAlchemy ORM
Stores user information, generated brands, logos, marketing content, sentiment analysis results, and chat history.

JWT Authentication System
Provides secure user registration, login, session management, and protected API access.

CORS Middleware
Enables secure cross-origin communication between frontend and backend services.

RESTful API Design
Provides modular API endpoints for authentication, brand generation, logo creation, content generation, sentiment analysis, and AI assistant functionalities.

PRE-REQUISITES:

Accounts & API Keys

Google AI Studio Account → https://aistudio.google.com
Hugging Face Account → https://huggingface.co

Tools & Frameworks

FastAPI → https://fastapi.tiangolo.com
Uvicorn Server → https://uvicorn.org

Python 3.10+ → https://python.org
SQLite Database → https://sqlite.org
Visual Studio Code → https://code.visualstudio.com
IBM Corporation, *watsonx.ai* → *https://developer.ibm.com/components/watsonx-ai*

AI Technologies

Google Gemini API
Stable Diffusion XL (SDXL) via Hugging Face

Frontend Technologies

HTML5
CSS3
JavaScript

Backend Libraries

SQLAlchemy
HTTPX
Pillow (PIL)
Passlib & BCrypt
Python-JOSE
Python-Dotenv

PROJECT WORKFLOW:

Phase 1: Environment Setup & Configuration

Activity 1.1: Set Up Python Environment and Install Dependencies

Activity 1.2: Configure Google Gemini API and Hugging Face API Access

Activity 1.3: Set Up SQLite Database and Environment Variables

Activity 1.4: Test API Connectivity and AI Model Responses

Phase 2: Backend Development

Activity 2.1: Set Up FastAPI Application Structure

Activity 2.2: Implement User Authentication (Signup/Login with JWT)

Activity 2.3: Develop Brand Name & Tagline Generation Module

Activity 2.4: Develop Logo Generation Module using Stable Diffusion XL

Activity 2.5: Develop Content Generation Module

Activity 2.6: Develop Sentiment Analysis Module

Activity 2.7: Develop AI Branding Assistant Module

Phase 3: Frontend Development

Activity 3.1: Design Responsive User Interface using HTML, CSS, and JavaScript

Activity 3.2: Integrate Frontend with FastAPI APIs

Activity 3.3: Develop Dashboard and Asset Management Features

Phase 4: Database Integration

Activity 4.1: Design Database Models using SQLAlchemy

Activity 4.2: Store User Data, Generated Brands, Logos, and Content

Activity 4.3: Implement Data Retrieval and Management Features

Phase 5: Deployment

Activity 5.1: Configure Application for Deployment

Activity 5.2: Deploy Backend and Frontend Services

Phase 6: Testing & Optimization

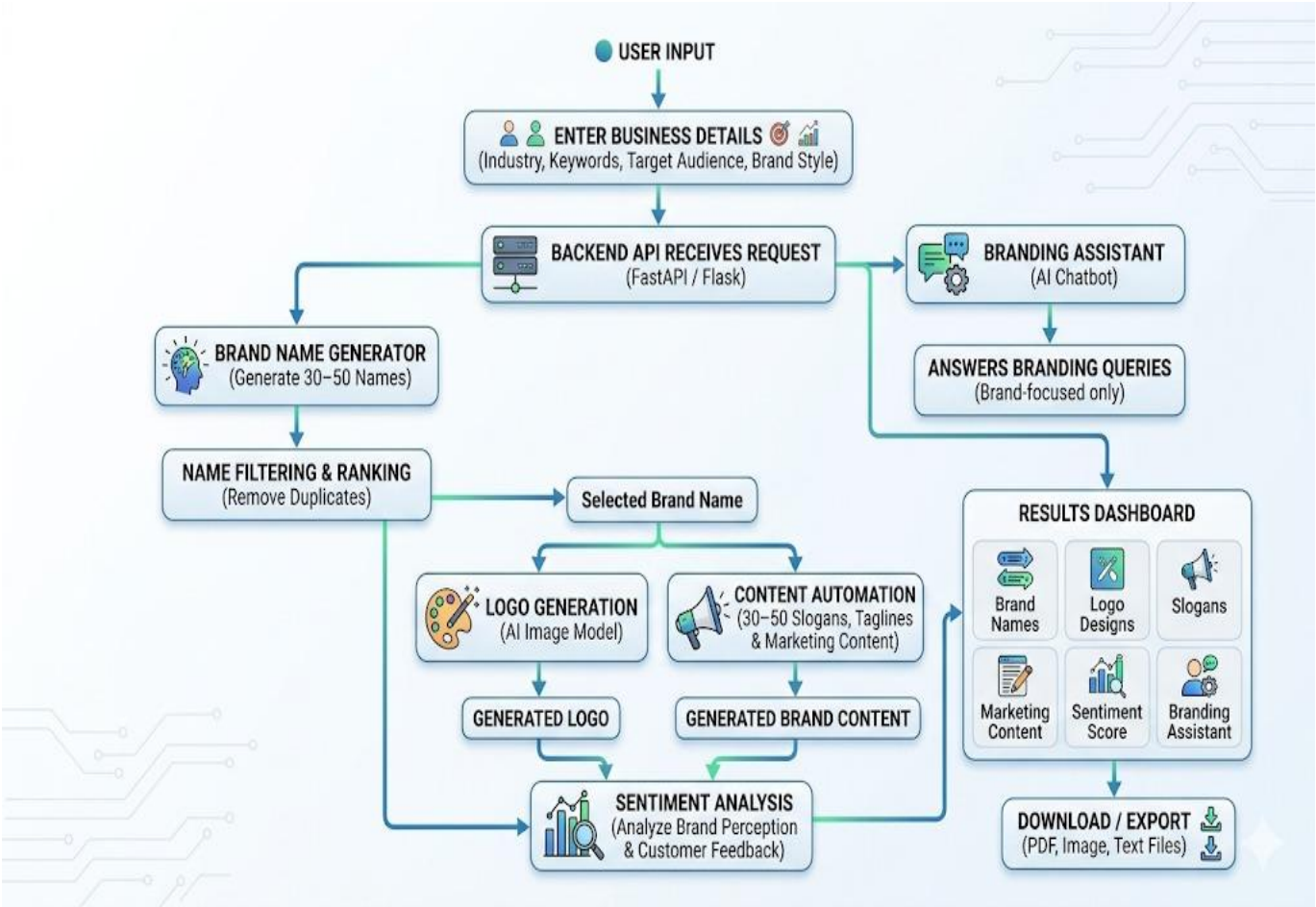
Activity 6.1: Functional Testing

Activity 6.2: API Testing and Validation

Activity 6.3: Performance Optimization

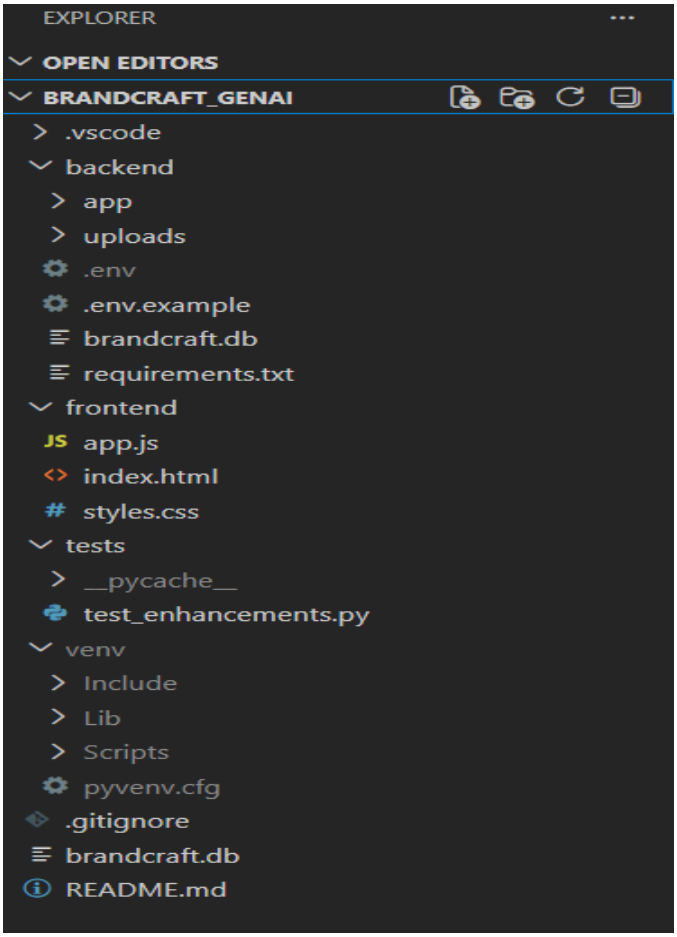
Activity 6.4: Bug Fixing and Final Evaluation

TECHNICAL ARCHITECTURE:



PROJECT STRUCTURE STEPUP:

Create the following directory structure



MILESTONE 1: Environment Setup & AI Service Configuration

This milestone establishes the foundational environment for BrandCraft by installing the required dependencies, configuring access to Google Gemini API and Hugging Face Stable Diffusion XL, and setting up the database and backend framework. Completing this milestone ensures that the application can generate brand names, taglines, marketing content, AI branding suggestions, and logos through integrated Generative AI services.

Activity 1.1: Install Python Environment & Required Dependencies

Before developing the branding platform, the development environment must be configured properly.

For Windows / Mac / Linux:

1. Install Python 3.10+

Download from:

https://www.python.org/downloads/

Create and activate a virtual environment.

python -m venv venv

venv\Scripts\activate

2. Install Core Project Dependencies

pip install fastapi uvicorn sqlalchemy python-dotenv google-generativeai httpx pillow passlib bcrypt python-jose

3. Verification

python --version

pip list

If the environment is properly configured, the package list should include:

fastapi
sqlalchemy
google-generativeai
httpx
pillow
python-jose
passlib

Activity 1.2: Configure Google Gemini API

BrandCraft uses **Google Gemini** as its primary Large Language Model (LLM) for brand generation, content creation, sentiment analysis, and AI branding assistance.

Steps:

1. Create or Sign In to Google AI Studio

https://aistudio.google.com

2. Generate a Gemini API Key

- Open Google AI Studio
- Navigate to API Keys
- Create New API Key
- Copy the generated key

3. Add the Key to the .env File

GEMINI_API_KEY="your_gemini_api_key"

4. Verify API Access

Execute a simple prompt request to ensure Gemini is responding correctly.

Example:

```
import google.generativeai as genai

genai.configure(api_key="your_gemini_api_key")

model = genai.GenerativeModel("gemini-pro")

response = model.generate_content(
    "Generate 3 creative brand names for a technology startup."
)

print(response.text)
```

Successful execution confirms Gemini connectivity.

Activity 1.3: Configure Hugging Face API for Stable Diffusion XL

BrandCraft uses **Stable Diffusion XL (SDXL)** through Hugging Face for AI-powered logo generation.

Steps:

1. Create or Sign In to Hugging Face

https://huggingface.co

2. Generate an Access Token

- Go to Settings → Access Tokens
- Click New Token
- Select Read Access
- Copy the generated token

3. Add Credentials to the .env File

```
HUGGINGFACE_API_KEY="your_huggingface_api_key"
```

4. Verify API Access

Visit:

https://huggingface.co/stabilityai/stable-diffusion-xl-base-1.0

Successful access confirms SDXL availability.

Activity 1.4: Configure SQLite Database & Backend Services

BrandCraft uses **SQLite** for storing user information, generated brands, logos, content, and chat history.

Steps:

1. Configure Database URL

Add to .env file:

DATABASE_URL=sqlite:///brandcraft.db

2. Initialize Database

Run the application to create database tables automatically through SQLAlchemy models.

3. Start FastAPI Server

uvicorn backend.app.main:app --reload

4. Verify Backend Availability

Open:

http://localhost:8000/docs

FastAPI Swagger UI should display all available APIs, including:

- Authentication APIs
- Brand Generation APIs
- Logo Generation APIs
- Content Generation APIs
- Sentiment Analysis APIs
- AI Assistant APIs

Successful access confirms that the backend, database, and AI integrations are configured correctly.

MILESTONE 2: Core Backend Development

Objective

Build the FastAPI-based backend infrastructure that powers all branding functionalities in BrandCraft. This milestone focuses on creating a modular API architecture, implementing user authentication, integrating Google Gemini and Stable Diffusion XL, connecting the SQLite database, and developing the core AI-powered branding features such as brand generation, logo creation, content generation, sentiment analysis, and AI branding assistance.

Activity 2.1: FastAPI Application Initialization

Create the backend project structure and configure FastAPI application setup.

Install Dependencies

Open terminal/command prompt and run:

python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt

```
PS C:\Users\hafee\.gemini\antigravity\scratch\brandcraft_genai\brandcraft_genai> python -m venv venv
>> venv\Scripts\activate
>> pip install -r requirements.txt
>>
```

Core Dependencies

pip install fastapi uvicorn sqlalchemy google-generativeai httpx pillow passlib bcrypt python-jose python-dotenv

Activity 2.2: Initialize the FastAPI Application

Configure the main FastAPI application in main.py.

backend > app > main.py

```python
import os
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse
from pathlib import Path

from app.database import engine, Base

from app.api import (
    auth_router, generator_router, logo_router,
    content_router, sentiment_router, assistant_router
)
```

The application performs:

- FastAPI app creation
- API router registration
- CORS middleware configuration
- Database initialization
- Environment variable loading
- Authentication setup

Activity 2.3: Implement User Authentication System

Develop secure user management features.

Authentication Features

- User Registration
- User Login
- JWT Token Generation
- Password Hashing using BCrypt
- Protected API Access
- User Verification

auth.py

backend > app > auth.py

```python
from datetime import datetime, timedelta
from typing import Optional, Union, Any
from jose import jwt, JWTError
import bcrypt
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session

from app.config import SECRET_KEY, ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES, REFRESH_TOKEN_EXPIRE_DAYS
from app.database import get_db
from app.models import User

# OAuth2 scheme for extraction of token
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/login", auto_error=False)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    try:
        return bcrypt.checkpw(plain_password.encode('utf-8'), hashed_password.encode('utf-8'))
    except Exception:
        return False

def get_password_hash(password: str) -> str:
    return bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')

def create_access_token(subject: Union[str, Any], expires_delta: Optional[timedelta] = None) -> str:
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)

    to_encode = {"exp": expire, "sub": str(subject)}
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt
```

```python
    return encoded_jwt

def create_refresh_token(subject: Union[str, Any], expires_delta: Optional[timedelta] = None) -> str:
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(days=REFRESH_TOKEN_EXPIRE_DAYS)

    to_encode = {"exp": expire, "sub": str(subject)}
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

def verify_token(token: str) -> Optional[str]:
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload.get("sub")
    except JWTError:
        return None

def get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)) -> User:
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    if not token:
        raise credentials_exception

    email = verify_token(token)
    if email is None:
        raise credentials_exception

    user = db.query(User).filter(User.email == email).first()
    if user is None:
        raise credentials_exception

    if not user.is_active:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Account is inactive. Please verify your email first."
        )

    return user
```

APIs
/api/auth/register
/api/auth/login
/api/auth/profile

Activity 2.4: Configure AI Service Integrations

Implement AI utility functions for external AI services.

Modules Implemented

Google Gemini Integration

- Brand Name Generation
- Tagline Generation
- Brand Story Generation
- Marketing Content Generation
- AI Branding Assistant

```python
from fastapi import APIRouter
import random

router = APIRouter(prefix="/api/tagline")

@router.post("/")
def generate_tagline(brand_name: str):
    taglines = [
        f"{brand_name} - Innovate Your Future",
        f"{brand_name} - Think Beyond",
        f"{brand_name} - Powering Ideas",
        f"{brand_name} - Where Vision Meets Reality"
    ]

    return {
        "tagline": random.choice(taglines)
    }
```

```python
from fastapi import APIRouter

router = APIRouter(prefix="/api/story")

@router.post("/")
def generate_story(brand_name: str, business_type: str):
    story = f"""
    {brand_name} was founded with a vision to transform the {business_type} industry.
    With innovation at its core, the company strives to deliver high-quality solutions
    and create a lasting impact on customers worldwide.
    """

    return {
        "brand_story": story
    }
```

```python
from fastapi import APIRouter

router = APIRouter(prefix="/api/story")

@router.post("/")
def generate_story(brand_name: str, business_type: str):
    story = f"""
    {brand_name} was founded with a vision to transform the {business_type} industry.
    With innovation at its core, the company strives to deliver high-quality solutions
    and create a lasting impact on customers worldwide.
    """

    return {
        "brand_story": story
    }
```

```python
from fastapi import APIRouter

router = APIRouter(prefix="/api/marketing")

@router.post("/")
def generate_marketing_content(brand_name: str, product: str):
    content = f"""
    Introducing {product} by {brand_name}! 🚀

    Experience innovation like never before.
    Designed to make your life easier, smarter, and better.

    Don't wait – try {brand_name} today!
    """

    return {
        "marketing_content": content
    }
```

```python
from fastapi import APIRouter

router = APIRouter(prefix="/api/assistant")

@router.post("/")
def branding_assistant(query: str):
    response = f"""
    AI Assistant Response:

    Based on your query: "{query}",
    I suggest focusing on strong branding, clear messaging,
    and customer engagement strategies.
    """

    return {
        "response": response
    }
```

Stable Diffusion XL Integration

- AI Logo Generation
- Brand Visual Creation

Activity 2.5: Brand Name & Tagline Generation Feature

Brand Generator (/api/generator)

Input: Industry, keywords, target audience, brand personality, and preferences.

Prompt Construction: Backend validates inputs and constructs optimized prompts for Gemini.

AI Call: Gemini generates brand names, taglines, and brand descriptions.

Output: Returns multiple brand names with taglines and branding suggestions.

Features

- Industry-specific naming
- Creative branding suggestions
- Tagline generation
- Brand meaning generation

```python
@router.post("/names", response_model=BrandResponse)
def generate_names(req: BrandRequest, current_user: User = Depends(get_current_user)):
    # Validate required fields
    missing = []
    if not req.business_type:
        missing.append('business_type')
    if not req.industry:
        missing.append('industry')
    if not req.target_audience:
        missing.append('target_audience')
    if not req.brand_personality:
        missing.append('brand_personality')
    if not req.preferred_language:
        missing.append('preferred_language')
    if not req.country:
        missing.append('country')
    if missing:
        raise HTTPException(status_code=400, detail=f"Missing required fields: {', '.join(missing)}")
    if not HAS_GEMINI_KEY:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Gemini API Key is not configured in backend/.env. Please configure GEMINI_API_KEY to generate brand names with AI."
        )
```

```python
    pool = name_pools.get(lang, name_pools["english"])
    meanings = meaning_templates.get(lang, meaning_templates["english"]).copy()
    tagline_tmpl = tagline_templates.get(lang, tagline_templates["english"])
    shuffle(meanings)

    brands = []
    used_names = set()
    for _ in range(10):
        while True:
            adj = random.choice(pool["adjectives"])
            noun = random.choice(pool["nouns"])
            name = adj + noun
            if name.lower() not in used_names:
                used_names.add(name.lower())
                break
        if not meanings:
            meanings = meaning_templates.get(lang, meaning_templates["english"]).copy()
            shuffle(meanings)
        meaning = meanings.pop().format(
            name=name,
            business_type=req.business_type,
            industry=req.industry,
            personality=req.brand_personality,
            audience=req.target_audience,
        )
        tagline = tagline_tmpl.format(personality=req.brand_personality, audience=req.target_audience)
        domains = [f"{name.lower()}.com", f"{name.lower()}.io", f"{name.lower()}.co"]
        brands.append({"name": name, "meaning": meaning, "tagline": tagline, "domains": domains})
    return {"brands": brands}
```

Activity 2.6: Content Generation Feature

Content Generator (/api/content)

Input: Brand information, content type, tone, and audience.

Prompt Construction: Backend prepares structured prompts for marketing content generation.

AI Call: Gemini generates customized content.

Output: Returns:

- Product descriptions
- Advertisement copy
- Social media captions
- Email marketing content
- Brand stories

```python
from fastapi import APIRouter

router = APIRouter(prefix="/api/content", tags=["Marketing Content"])

@router.post("/")
def generate_content(brand_name: str, product: str):
    content = f"""
    🚀 Introducing {product} by {brand_name}!

    Transform your experience with innovation and quality.
    Designed for people who demand the best.

    👉 Choose {brand_name} today and upgrade your lifestyle!
    """

    return {
        "brand_name": brand_name,
        "product": product,
        "marketing_content": content
    }
```

Activity 2.7: Sentiment Analysis Feature

Sentiment Analysis (/api/sentiment)

Input: Customer reviews or feedback text.

Processing: Backend analyzes sentiment and emotional context.

AI Call: Gemini processes review data.

Output: Returns:

- Positive sentiment
- Negative sentiment
- Neutral sentiment
- Customer perception summary
- Key insights

Activity 2.8: AI Branding Assistant Feature

Branding Assistant (/api/assistant)

Input: User branding queries and consultation requests.

Contextualization: Backend constructs branding-focused prompts.

AI Call: Gemini responds as an intelligent branding consultant.

Output: Returns:

- Branding recommendations
- Marketing suggestions
- Brand strategy guidance
- Design recommendations

```python
def analyze_lexicon_sentiment(text: str) -> SentimentResponse:
    """Performs rule-based sentiment and emotion analysis on review text."""
    logger.info("Using local lexicon-based sentiment engine.")

    # Clean up text and split into sentences
    sentences = [s.strip() for s in re.split(r'[.!?\n]', text.lower()) if s.strip()]

    # Detailed stems
    pos_stems = [
        "lov", "great", "excel", "best", "happ", "satisf", "amaz", "good", "wonder", "perfect",
        "awesom", "fast", "eas", "beaut", "stellar", "friend", "help", "thank", "super", "outstand",
        "impress", "recommend", "delight", "effic", "nice", "cool", "smart", "glad", "speed", "quick"
    ]
    neg_stems = [
        "bad", "worst", "terrib", "hate", "angr", "frustrat", "poor", "slow", "crash", "error",
        "issue", "bug", "brok", "expens", "useless", "garbag", "fail", "disappoint", "regret",
        "annoy", "pain", "horrib", "diffic", "rude", "delay", "wast", "dirt", "broke", "stupid"
    ]

    # Emotion stems
    emotions_stems = {
        "happy": ["happ", "glad", "lov", "joy", "smile", "delight", "pleas", "wonder", "perfect", "nice"],
        "angry": ["angr", "mad", "rage", "hate", "furi", "terrib", "garbag", "annoy", "rude", "stupid"],
        "excited": ["excit", "thrill", "awesom", "amaz", "spectac", "wow", "fantast", "super", "epic"],
        "frustrated": ["frustrat", "slow", "wast", "useless", "brok", "bugg", "crash", "stuck", "fail", "delay"],
        "satisfied": ["satisf", "content", "work", "fine", "help", "resolv", "recommend", "good", "okay"]
    }

    pos_sentences = 0
    neg_sentences = 0
    neu_sentences = 0
    total_pos_words = 0
    total_neg_words = 0

    emotion_scores = {"happy": 0.0, "angry": 0.0, "excited": 0.0, "frustrated": 0.0, "satisfied": 0.0}
    all_clean_words = []
```

Activity 2.9: Logo Generation Feature

Logo Generator (/api/logo)

Input: Brand name, industry, logo style, and color preferences.

Prompt Construction: Backend creates detailed logo generation prompts.

AI Call: Stable Diffusion XL generates logo images.

Output: Returns AI-generated logo designs and branding visuals.

Features

- Multiple logo styles
- Custom branding prompts
- Downloadable logo assets

```python
async def generate_sd_logo(prompt: str, seed: int) -> bytes:
    """Queries Hugging Face serverless Stable Diffusion XL endpoint."""
    headers = {
        "Authorization": f"Bearer {HUGGINGFACE_API_KEY}",
        "X-Use-Cache": "false"
    }
    payload = {
        "inputs": prompt,
        "parameters": {
            "seed": seed
        }
    }

    async with httpx.AsyncClient(timeout=45.0) as client:
        response = await client.post(HF_SDXL_URL, headers=headers, json=payload)
        if response.status_code != 200:
            raise Exception(f"HF API Error: {response.text}")
        return response.content

async def generate_single_logo_image(i: int, req: LogoRequest) -> bytes:
    # Directly use procedural Pillow generator to load instantly (under 10ms per logo)
    loop = asyncio.get_running_loop()
    img = await loop.run_in_executor(
        None,
        draw_procedural_logo,
        req.brand_name,
        req.industry,
        req.style,
        req.colors,
        req.logo_type,
        i
    )
    buffer = io.BytesIO()
    img.save(buffer, "PNG")
    return buffer.getvalue()
```

Activity 2.10: Database Integration

Configure SQLite database using SQLAlchemy ORM.

Database Operations

- Store User Information
- Store Generated Brands
- Store Generated Logos
- Store Marketing Content
- Store Chat History
- Manage User Assets

backend > app > database.py

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker
from app.config import DATABASE_URL

# Connect to database. In SQLite, check_same_thread=False allows multi-thread access
engine = create_engine(
    DATABASE_URL, connect_args={"check_same_thread": False} if DATABASE_URL.startswith("sqlite") else {}
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

Activity 2.11: REST API Development

Develop modular RESTful API endpoints for all application features.

API Modules

/api/auth
/api/generator
/api/logo
/api/content
/api/sentiment
/api/assistant

These APIs enable seamless communication between frontend and backend services.

Activity 2.12: System Setup & Initialization

Define application startup and deployment configuration.

Application Run

if __name__ == "__main__":

Configuration

- Start FastAPI using Uvicorn
- Configure host and port settings
- Initialize database connections
- Load AI service configurations

Deployment

The backend becomes accessible through:

http://localhost:8000

and provides all core BrandCraft services including authentication, brand generation, logo creation, content generation, sentiment analysis, and AI branding assistance.

```
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

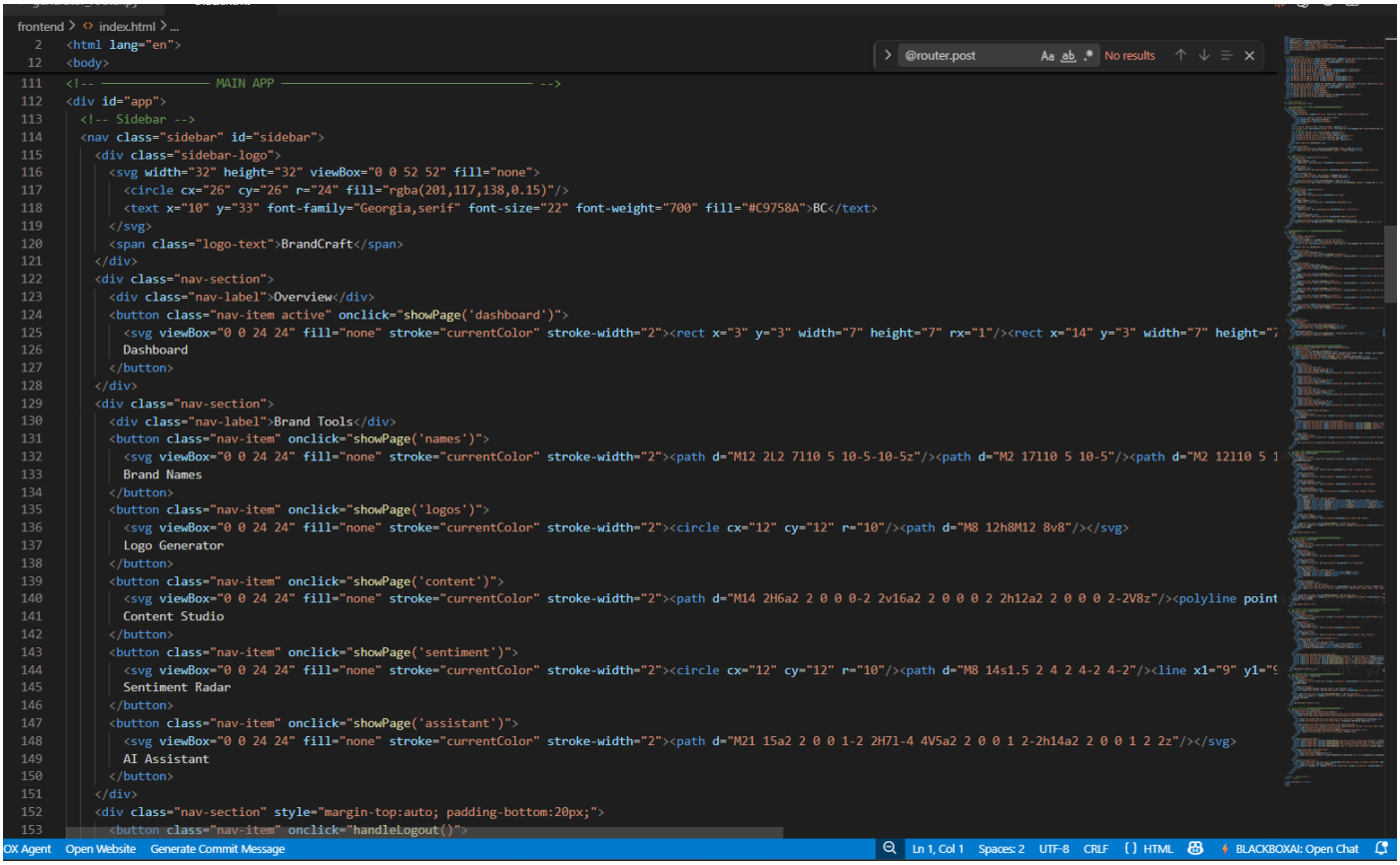
```
PS C:\Users\hafee\.gemini\antigravity\scratch\brandcraft_genai\brandcraft_genai> cd c:\Users\hafee\.gemini\antigravity\scratch\brandcraft_genai\brandcraft_genai\backend
>> python -m uvicorn app.main:app --reload
>>
INFO:     Will watch for changes in these directories: ['C:\\Users\\hafee\\.gemini\\antigravity\\scratch\\brandcraft_genai\\brandcraft_genai\\backend']
C:\Users\hafee\.gemini\antigravity\scratch\brandcraft_genai\brandcraft_genai\backend\app\api\generator_router.py:6: FutureWarning:

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

  import google.generativeai as genai
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [28516] using WatchFiles
C:\Users\hafee\.gemini\antigravity\scratch\brandcraft_genai\brandcraft_genai\backend\app\api\generator_router.py:6: FutureWarning:

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

  import google.generativeai as genai
INFO:     Started server process [28588]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

MILESTONE 3: Frontend Development – UI/UX Design

Activity 3.1: Base Template Structure (index.html)

Description

The index.html file serves as the main landing page of the BrandCraft platform. It introduces users to the application, highlights key features, and provides easy navigation to branding tools.



Base Template Features

- Gradient Background
  A full-page modern gradient (purple–blue) is implemented to give a premium and professional branding appearance.

- Responsive Navbar
  A sticky navigation bar positioned at the top includes:
  - Language selector dropdown
  - Theme toggle button (light/dark mode)

- Gradient Logo Text
  The application name “BrandCraft” is displayed using bold gradient-filled text to enhance visual appeal.

- Hero Section
  A prominent section containing:
  - Large headline introducing AI branding
  - Supporting subtext
  - Call-to-Action (CTA) button directing users to the branding tools page

- Interactive Feature Cards
  Hover-animated cards representing core functionalities:

  - Brand Name Generator
  - Logo Generator
  - Marketing Content Generator
  - Design System Generator
  - Sentiment Analysis
  - AI Branding Assistant

- Modern Grid Layout
  A responsive auto-fit grid system ensures proper alignment and display across all screen sizes.
- Info Section (Why Choose BrandCraft?)
  A clean white-background section explaining the benefits of the platform with structured feature tiles.
- Feature Highlights
  Dedicated tiles showcasing:
  - AI-powered branding tools
  - Instant content generation
  - Multilingual support
  - Smart automation
  - Professional output quality
- Footer
  A minimal centered footer displaying:

Along with model credits such as:

  - Gemini AI
  - OpenAI / LLM APIs
  - Stable Diffusion (for logo ideas)

- Theme Toggle
  A single-button toggle enabling users to switch between light and dark modes.
- Language Selector
  Dropdown menu supporting:
  - English
  - Hindi

    - Spanish
    - French
    - German

- Smooth UI Interactions
  Includes:
    - Hover animations
    - Shadows
    - Transitions
    - Smooth scrolling effects
- Mobile Optimization
  Fully responsive design ensuring compatibility with mobile, tablet, and desktop devices.

Activity 3.2: Implementation of Functional UI Blocks

Description

In this project, all branding functionalities are implemented within the index.html file as separate functional blocks (sections). Each block corresponds to a specific AI-powered feature and interacts with the backend APIs dynamically.

Implemented UI Blocks

1. Brand Name Generator Block

This section allows users to generate creative brand names.

- Input Fields:
    - Keywords
    - Industry (optional)
- Functionality:
    - User inputs data and clicks generate
    - A JavaScript function sends a request to the backend API
    - The generated brand names are displayed dynamically

2. Logo Generator Block

This section provides AI-based logo suggestions.

- Input Fields:
    - Brand name
    - Keywords
- Functionality:
    - Sends user input to backend
    - Displays logo ideas or descriptions returned by AI

3. Content Generator Block

This section generates marketing content for the brand.

- Input Fields:
    - Brand name
    - Product/service description
    - Tone (optional)
- Functionality:
    - Calls content generation API
    - Displays outputs such as:
        - Ad copy
        - Social media captions
        - Descriptions

4. Sentiment Analysis Block

This section analyzes the sentiment of user-provided text.

- Input Fields:
    - Customer review / feedback text
- Functionality:
    - Sends text to backend API
    - Displays:
        - Sentiment (Positive/Negative/Neutral)
        - Improved or refined version of text

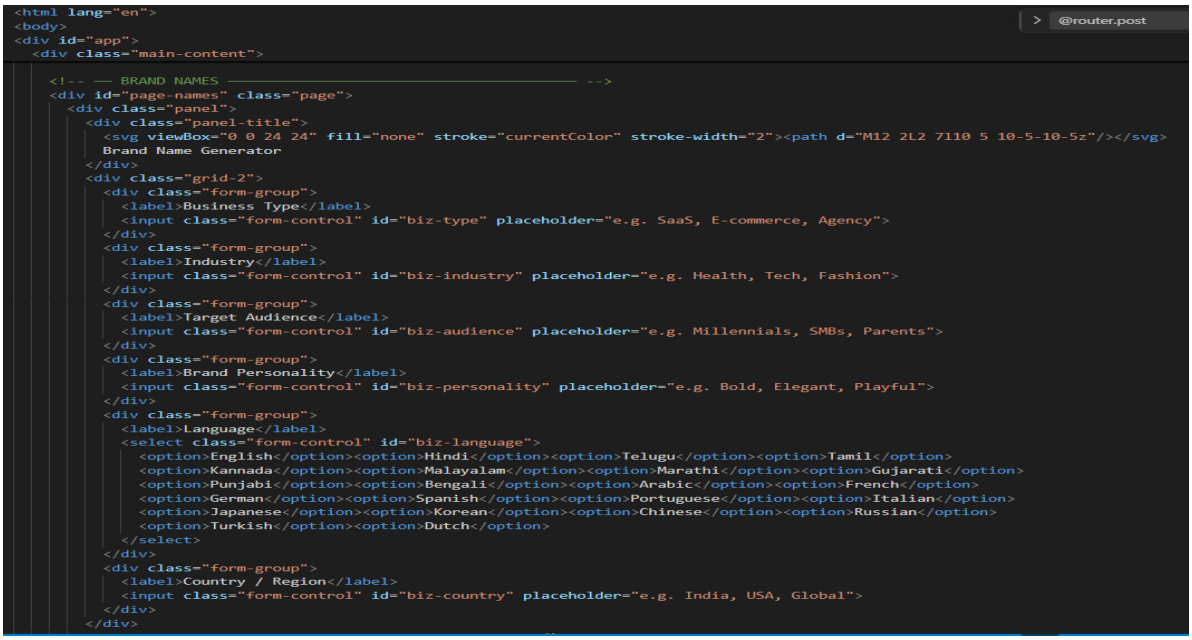
5. AI Branding Assistant Block

This section acts as an interactive chatbot for branding guidance.

- Input Fields:
    - User query/message
- Functionality:
    - Sends query to AI backend
    - Displays intelligent responses for:
        - Branding strategies
        - Naming ideas
        - Marketing suggestions

Technical Implementation

- Each block is implemented as a separate section in index.html
- JavaScript is used to:
    - Capture user inputs
    - Send API requests (using fetch/axios)
    - Dynamically update results without reloading the page
- The UI is designed to be responsive and interactive



```
<!-- ── LOGO GENERATOR ──────────────────────────────── -->
<div id="page-logos" class="page">
  <div class="panel">
    <div class="panel-title">
      <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="12" cy="12" r="10"/><path d="M8 12h8M12 8v8"/></svg>
      Logo Concept Generator
    </div>
    <div class="grid-2">
      <div class="form-group">
        <label>Brand Name</label>
        <input class="form-control" id="logo-brand" placeholder="e.g. NovaSpark">
      </div>
      <div class="form-group">
        <label>Industry</label>
        <input class="form-control" id="logo-industry" placeholder="e.g. Technology">
      </div>
      <div class="form-group">
        <label>Logo Style</label>
        <select class="form-control" id="logo-style">
          <option>Minimal</option><option>Luxury</option><option>Modern</option>
          <option>Vintage</option><option>Startup</option><option>Tech</option>
          <option>Fashion</option><option>Mascot</option>
        </select>
      </div>
      <div class="form-group">
        <label>Color Palette</label>
        <div style="display: flex; gap: 8px;">
          <input class="form-control" id="logo-colors" placeholder="e.g. Navy & Gold, Pastels" style="flex: 1;">
          <input type="color" id="logo-color-picker" value="#C9758A" style="width: 42px; height: 42px; padding: 0; border: 1px solid var(--card-border); border-r
        </div>
      </div>
    </div>
    <button class="btn btn-rose" onclick="generateLogos()" id="btn-gen-logos">
      <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><rect x="3" y="3" width="18" height="18" rx="2"/><circle
      Generate Logo Concepts
    </button>
  </div>
  <div id="logos-results"></div>
</div>
```

```
<!-- ── CONTENT STUDIO ──────────────────────────────── -->
<div id="page-content" class="page">
  <div class="panel">
    <div class="panel-title">
      <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline po
      Content Automation Studio
    </div>
    <div class="grid-2">
      <div class="form-group">
        <label>Brand Name</label>
        <input class="form-control" id="cont-brand" placeholder="Your brand name">
      </div>
      <div class="form-group">
        <label>Industry</label>
        <input class="form-control" id="cont-industry" placeholder="e.g. Fitness, Food, Finance">
      </div>
      <div class="form-group" style="grid-column:1/-1;">
        <label>Brand Tone</label>
        <select class="form-control" id="cont-tone">
          <option>Professional & Trustworthy</option><option>Fun & Playful</option>
          <option>Bold & Inspiring</option><option>Elegant & Luxurious</option>
          <option>Friendly & Approachable</option><option>Witty & Creative</option>
        </select>
      </div>
    </div>
    <div style="display:flex;gap:10px;flex-wrap:wrap;">
      <button class="btn btn-rose" onclick="generateContent('slogans')" id="btn-slogans">✨ Slogans</button>
      <button class="btn btn-outline" onclick="generateContent('stories')" id="btn-stories">📖 Brand Stories</button>
      <button class="btn btn-outline" onclick="generateContent('social')" id="btn-social">📱 Social Captions</button>
      <button class="btn btn-outline" onclick="generateContent('ads')" id="btn-ads">📢 Ad Copies</button>
      <button class="btn btn-outline" onclick="generateContent('emails')" id="btn-emails">📧 Email Templates</button>
    </div>
  </div>
  <div id="content-results"></div>
</div>

<!-- ── SENTIMENT RADAR ──────────────────────────────── -->
```

```
<!-- ── SENTIMENT RADAR ──────────────────────────────── -->
<div id="page-sentiment" class="page">
  <div class="panel">
    <div class="panel-title">
      <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="12" cy="12" r="10"/><path d="M8 14s1.5 2 4 2 4-2 4-2"/></svg>
      Sentiment Radar
    </div>
    <div class="form-group">
      <label>Paste Customer Reviews (one per line, or as a block)</label>
      <textarea class="form-control" id="sentiment-input" rows="6" placeholder="The product is amazing! Delivery was fast.&#10;Terrible quality, very disappointed.&#
    </div>
    <button class="btn btn-rose" onclick="analyzeSentiment()" id="btn-sentiment">
      <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="11" cy="11" r="8"/><line x1="21" y1="21" x2="16.
      Analyze Sentiment
    </button>
  </div>
  <div id="sentiment-results"></div>
</div>
```

```
<!-- ── AI ASSISTANT ──────────────────────────────────── -->
<div id="page-assistant" class="page">
  <div class="panel" style="padding:0;overflow:hidden;">
    <div style="padding:20px 24px 16px;border-bottom:1px solid var(--card-border);display:flex;align-items:center;gap:12px;">
      <div style="width:40px;height:40px;border-radius:50%;background:linear-gradient(135deg,#F8C8D4,#FDCFB0);display:flex;align-items:center;justify-content:ce
      <div>
        <div style="font-weight:600;font-size:15px;color:var(--text-dark);">BrandCraft AI Assistant</div>
        <div style="font-size:12px;color:var(--text-light);">Branding & Marketing specialist</div>
      </div>
      <div style="margin-left:auto;display:flex;align-items:center;gap:12px;">
        <button id="btn-clear-chat" class="btn btn-outline" style="padding: 4px 8px; font-size: 11px; height: auto;" onclick="clearChatHistory()">Clear History</but
        <div style="display:flex;align-items:center;gap:6px;">
          <div style="width:8px;height:8px;background:#67C23A;border-radius:50%;"></div>
          <span style="font-size:12px;color:var(--text-light);">Online</span>
        </div>
      </div>
    </div>
    <div class="suggested-prompts">
      <button class="prompt-chip" onclick="sendPromptChip('How do I build a strong brand identity?')">Brand identity tips</button>
      <button class="prompt-chip" onclick="sendPromptChip('What makes a great brand name?')">Great brand names</button>
      <button class="prompt-chip" onclick="sendPromptChip('How to create a brand strategy for a startup?')">Startup strategy</button>
      <button class="prompt-chip" onclick="sendPromptChip('Tips for social media branding')">Social media branding</button>
    </div>
    <div class="chat-window" id="chat-window">
      <div class="chat-msg assistant">
        <div class="chat-name">BrandCraft AI</div>
        <div class="chat-bubble">Hello! I'm your BrandCraft AI Assistant 👋<br><br>I specialize in branding, marketing, content strategy, logo design, and business
      </div>
    </div>
    <div class="chat-input-row">
      <input class="chat-input" id="chat-input" placeholder="Ask about branding, strategy, marketing…" onkeydown="if(event.key==='Enter')sendChat()">
      <button class="chat-send" onclick="sendChat()">
        <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><line x1="22" y1="2" x2="11" y2="13"/><polygon points="22
      </button>
    </div>
  </div>
</div>
```

MILESTONE 4: Deployment

Activity 4.1: Deployment Preparation

Local Deployment

To run the BrandCraft – Generative AI Branding Platform locally, follow these steps:

1. Open terminal in your backend project folder
2. Activate virtual environment:

venv\Scripts\activate

3. Install required dependencies:

pip install -r requirements.txt

4. Run the FastAPI server using Uvicorn:

uvicorn main:app --reload

```
PS C:\Users\hafee\.gemini\antigravity\scratch\brandcraft_genai\brandcraft_genai> cd c:\Users\hafee\.gemini\antigravity\scratch\brandcraft_genai\brandcraft_genai\backend
>> python -m uvicorn app.main:app --reload
>>
INFO:     Will watch for changes in these directories: ['C:\\Users\\hafee\\.gemini\\antigravity\\scratch\\brandcraft_genai\\brandcraft_genai\\backend']
C:\Users\hafee\.gemini\antigravity\scratch\brandcraft_genai\brandcraft_genai\backend\app\api\generator_router.py:6: FutureWarning:

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

  import google.generativeai as genai
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [28516] using WatchFiles
C:\Users\hafee\.gemini\antigravity\scratch\brandcraft_genai\brandcraft_genai\backend\app\api\generator_router.py:6: FutureWarning:

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

  import google.generativeai as genai
INFO:     Started server process [28588]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

MILESTONE 5: Testing & Optimization

Activity 5.1: Functional Testing

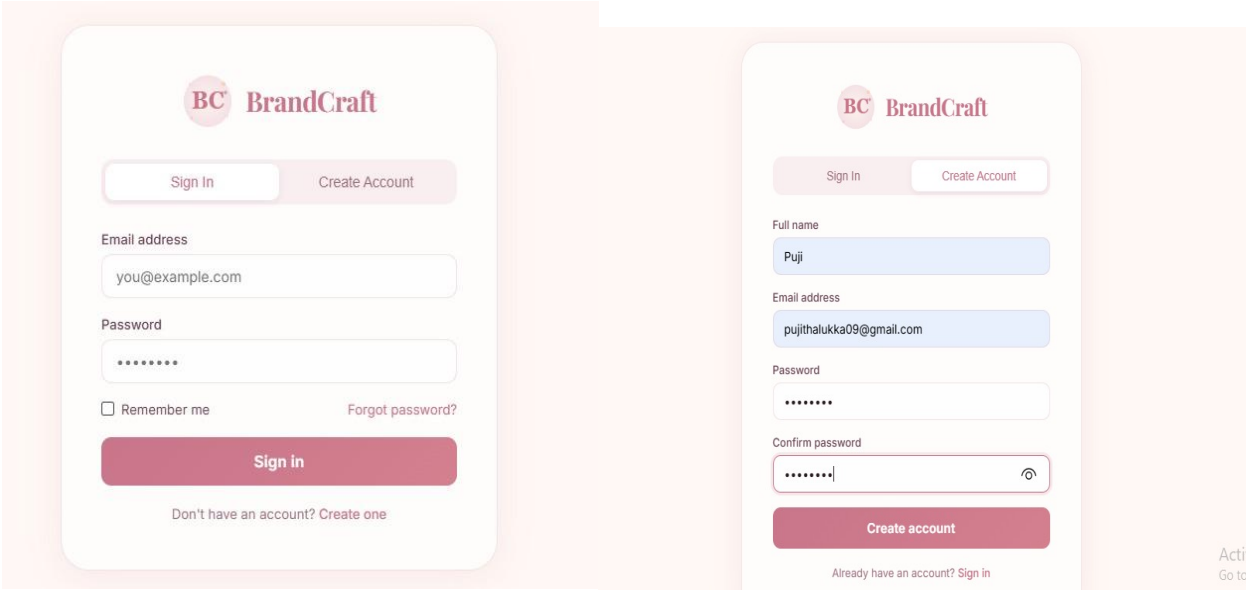
Test Cases:

Test 1: Login / Signup Page

1. Open the application
2. Navigate to Signup/Login section
3. Enter valid details (username, email, password)
4. Click "Signup" or "Login"

Output:

- User account is created successfully (Signup)
- User is logged in successfully (Login)
- Redirected to dashboard



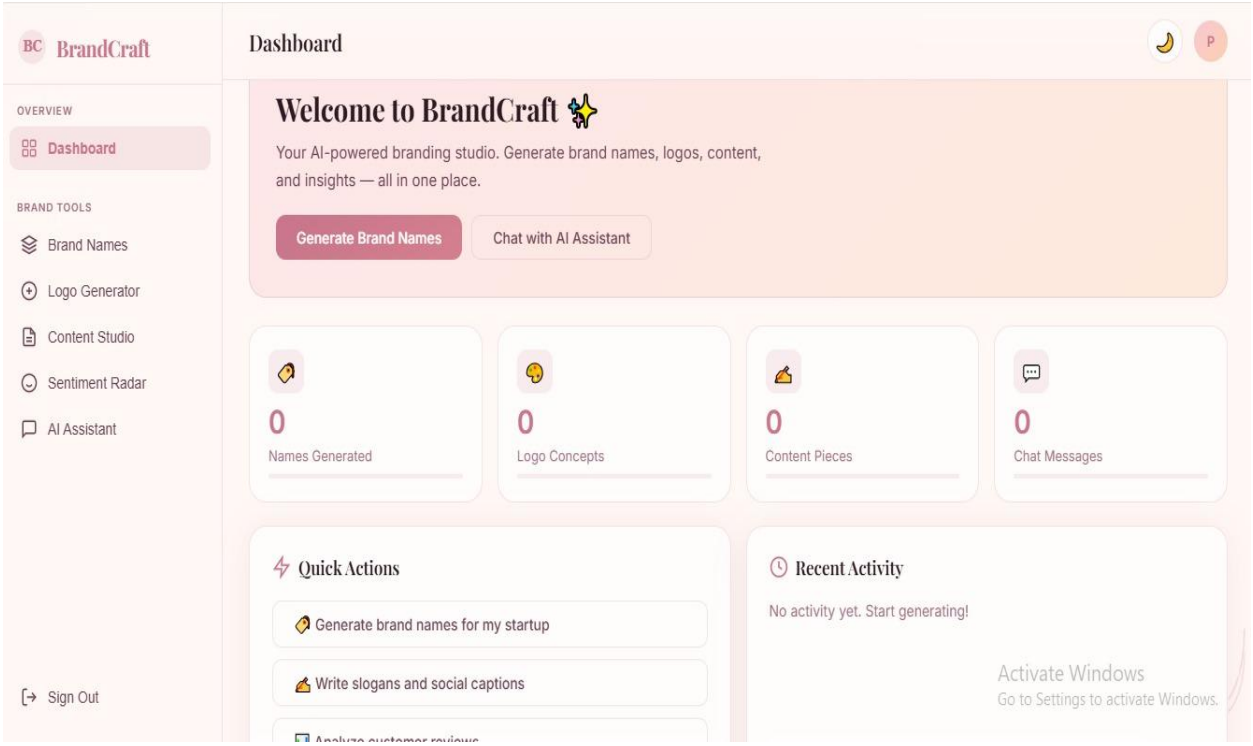
Test 2: Dashboard

1. After login, user lands on dashboard
2. View available features/sections:
    - Brand Name Generator
    - Logo Generator
    - Content Generator
    - Sentiment Analyzer
    - AI Assistant

Output:

- Dashboard loads correctly

- All feature blocks are visible
- Navigation between sections works smoothly

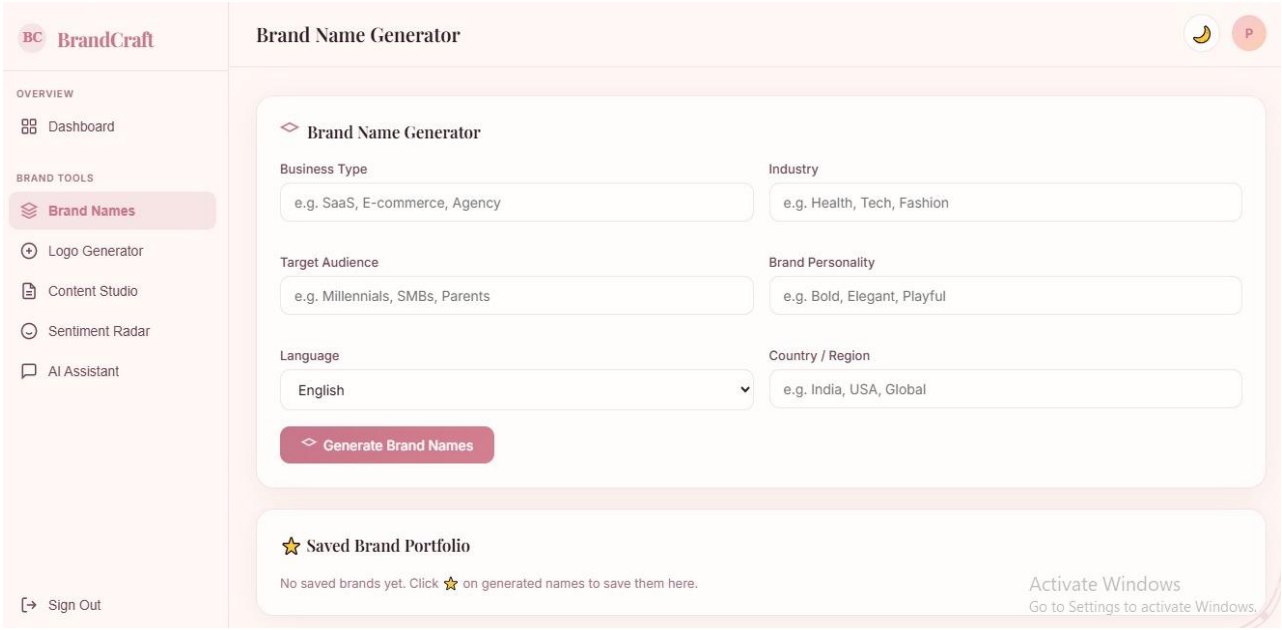


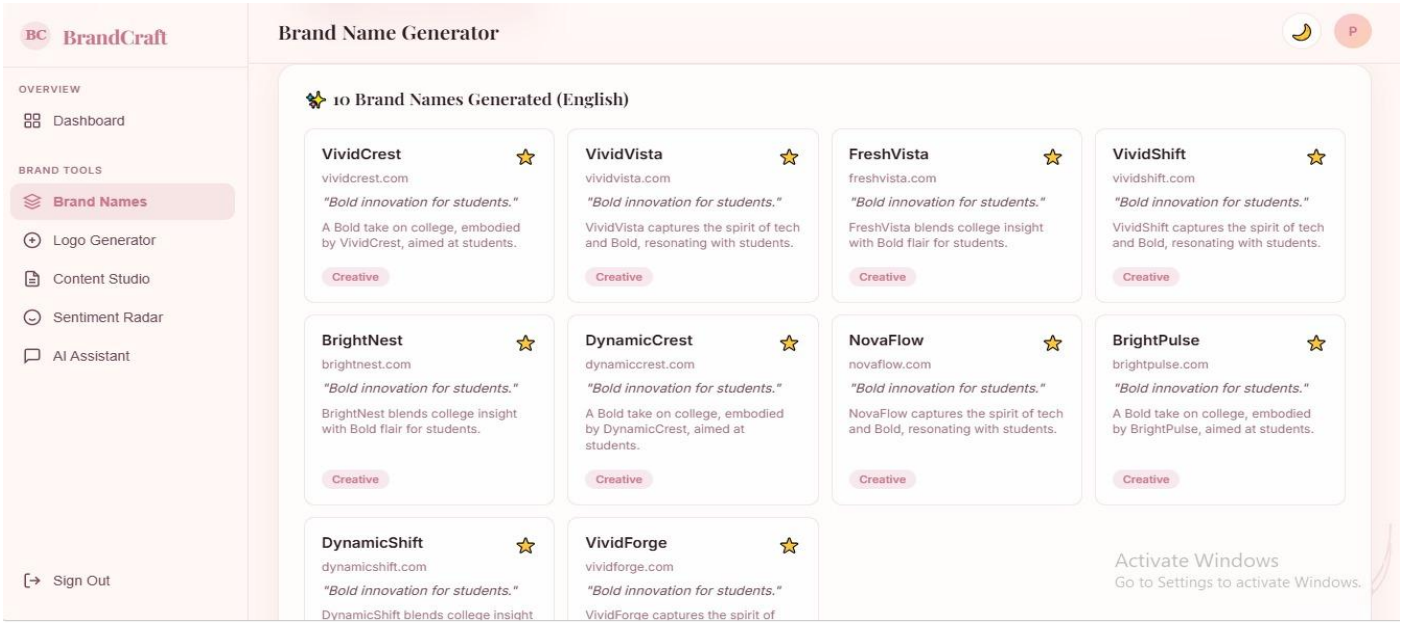
Test 3: Brand Name Generator

1. Navigate to Brand Name Generator section
2. Enter input: "Smart, innovative"
3. Click "Generate Brand Names"

Output:

- AI-generated brand names are displayed
- Results appear dynamically without refresh



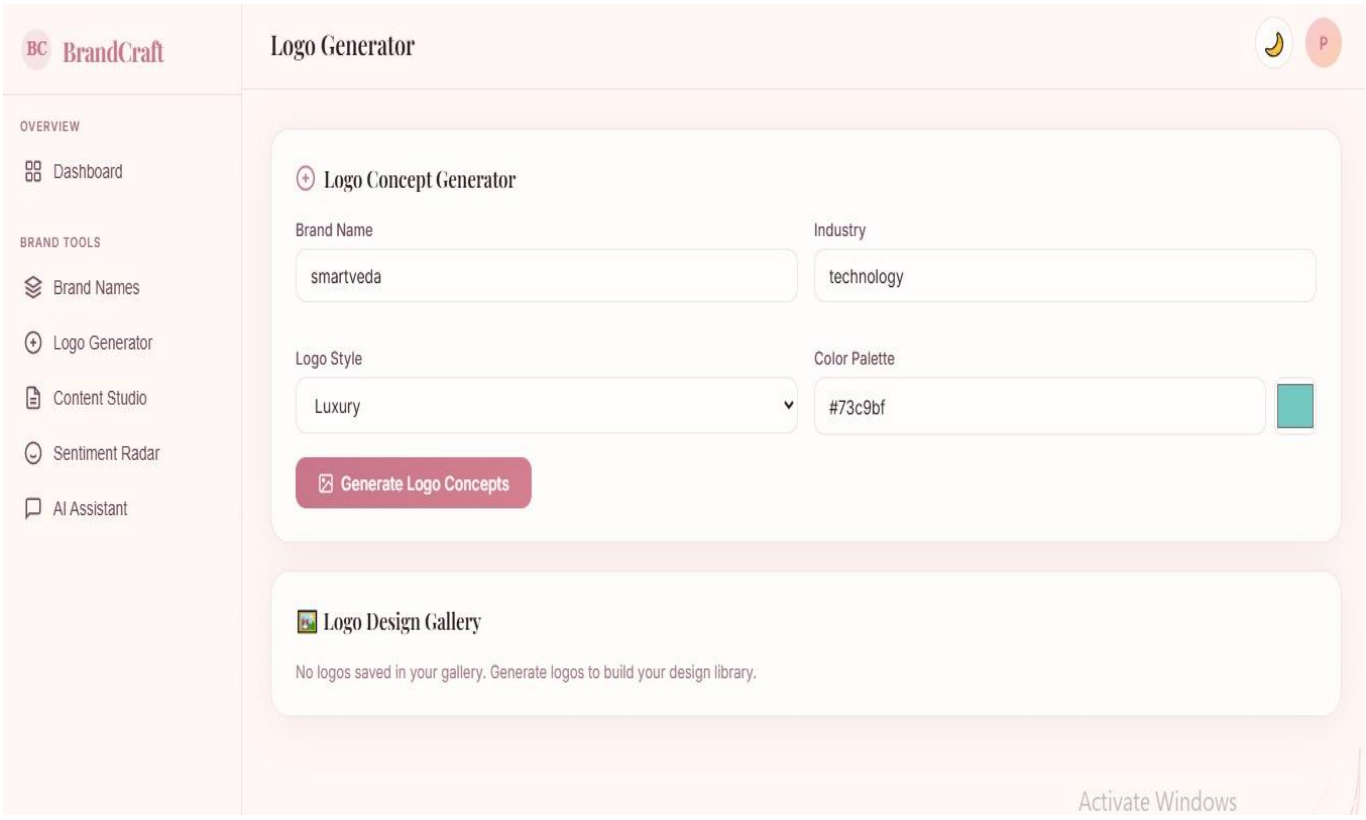


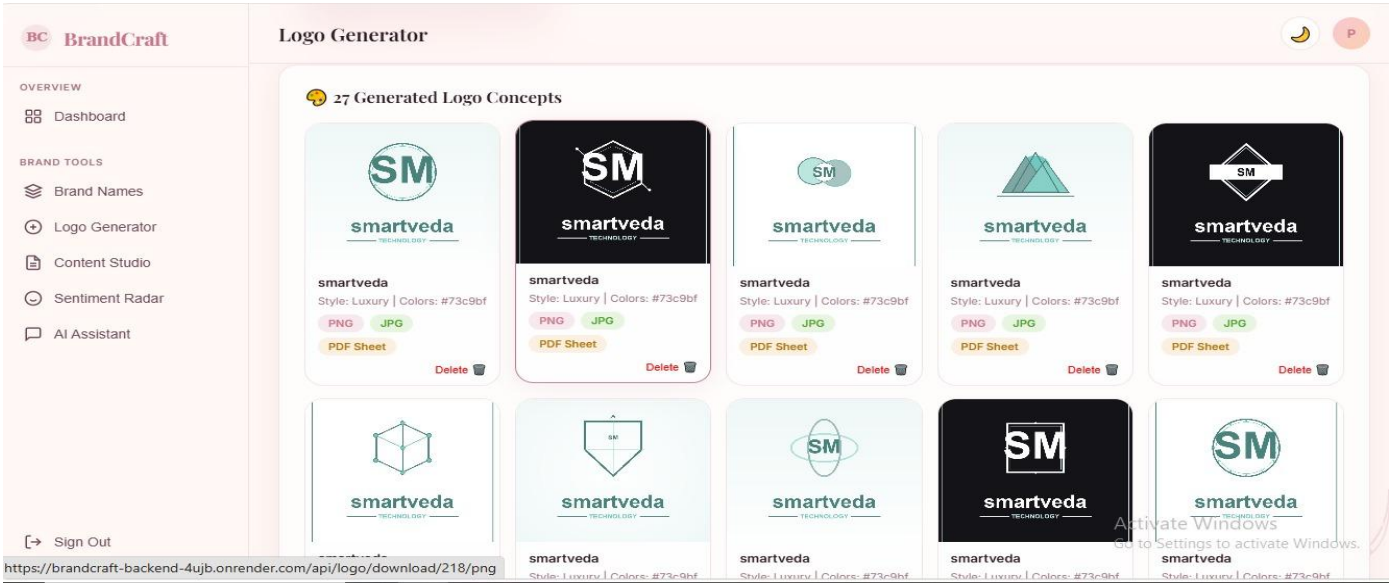
Test 4: Logo Generator

1. Navigate to Logo Generator section
2. Enter: "Technision, technology, modern, cool, attractive"
3. Click "Generate Logo"

Output:

- Logo ideas/descriptions are generated
- Output is displayed properly



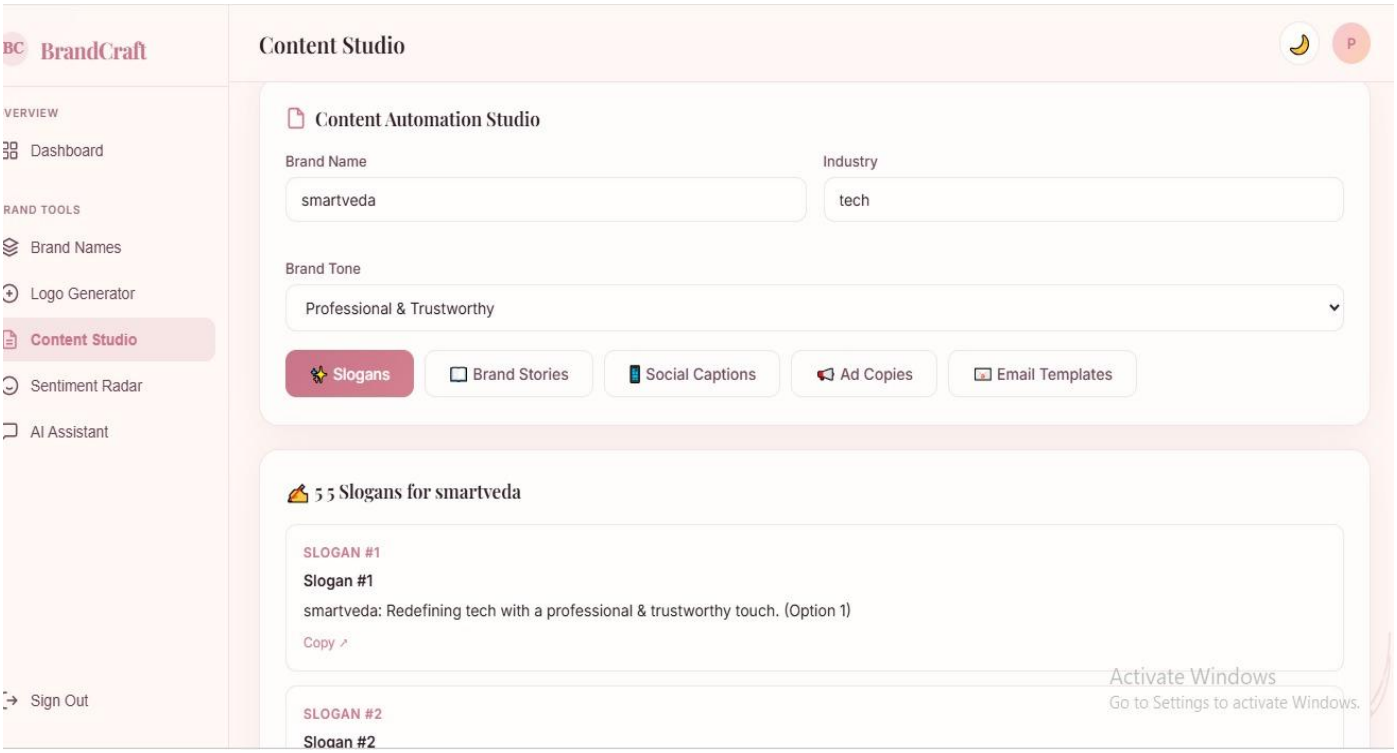


Test 5: Content Studio

1. Navigate to Content section
2. Enter: "A brand where you find wide variety of cool clothes"
3. Click "Generate Content"

Output:

- Marketing content generated:
    - Ad copy
    - Captions
    - Descriptions
- Output displayed clearly

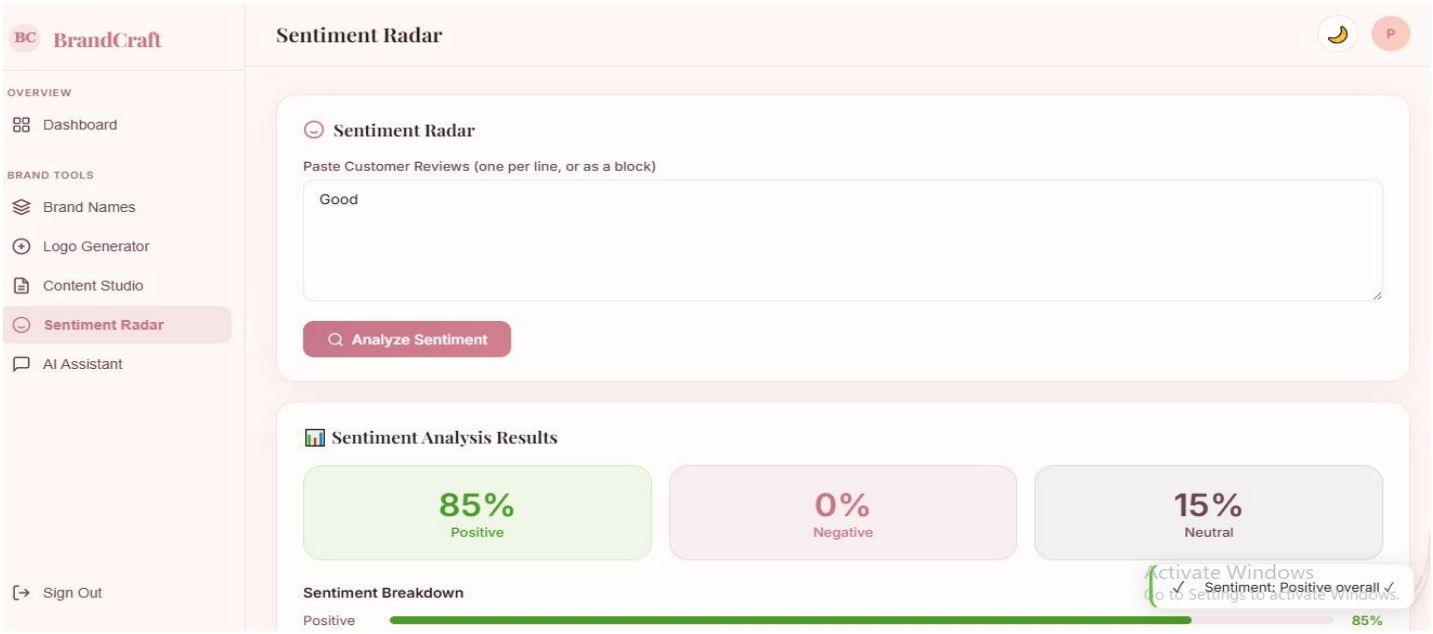


Test 6: Sentiment Analyzer

1. Navigate to Sentiment section
2. Enter: "It is good and nice"
3. Click "Analyze & Rewrite"

Output:

- Sentiment result: Positive
- Improved text is displayed

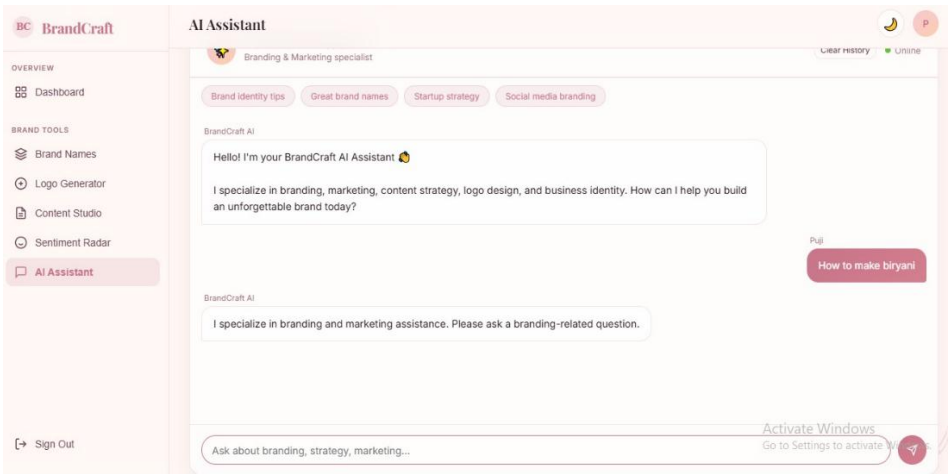


Test 7: AI Branding Assistant

1. Navigate to AI Assistant section
2. Example Input: "What is a brand?"
3. Click "Send"

Output:

- AI-generated response is displayed
- Chat-like interaction works properly



Activity 5.2: Optimization

Performance Optimization

- Reduced unnecessary API calls
- Efficient handling of responses using JavaScript
- Fast loading of frontend components

UI/UX Optimization

- Improved layout spacing and alignment
- Added hover effects and transitions
- Ensured responsive design across devices

Error Handling

- Display messages for invalid or empty inputs
- Handled API errors gracefully
- Ensured smooth user experience.

CONCLUSION:

The **BrandCraft – Generative AI Branding Platform** successfully demonstrates how modern AI technologies can be leveraged to simplify and automate real-world branding processes. By integrating a FastAPI-based backend with an interactive single-page frontend, the project delivers a seamless user experience for generating brand names, logo ideas, marketing content, sentiment analysis, and AI-driven branding assistance.

The system effectively combines multiple AI-powered functionalities into a unified platform, allowing users to move from ideation to content creation within a single interface. The implementation of authentication (login/signup), dashboard navigation, and modular UI blocks ensures both usability and scalability. The use of API-based architecture enabled smooth communication between frontend and backend, while dynamic rendering enhanced performance and responsiveness.

Throughout the development process, key challenges such as API integration, handling asynchronous requests, and ensuring proper frontend–backend connectivity were addressed. This contributed to a deeper understanding of full-stack development, RESTful APIs, and AI model integration in practical applications.
The project also highlights the importance of clean UI/UX design, efficient error handling, and structured code organization in building a reliable application. By adopting a modular block-based approach within a single-page interface, the system maintains simplicity without compromising functionality.

Looking ahead, BrandCraft can be further enhanced by adding features such as real-time logo generation, advanced personalization, user project saving, analytics dashboards, and deployment on scalable cloud platforms. With these improvements, it has strong potential to evolve into a fully functional commercial AI branding assistant.
Overall, this project showcases the effective use of Generative AI in creative automation and serves as a strong foundation for developing intelligent, user-centric digital solutions.